

Lecture 3

Machine Learning Mechanics – Terminology and Techniques

Framing ML Problems and ML Techniques and Terminologies

EECS 189/289, Fall 2025 @ UC Berkeley

Joseph E. Gonzalez and Narges Norouzi

Do not edit
How to change the
design



**Join at slido.com
#2826097**

① The Slido app must be installed on every computer you're presenting from

slido



2826097

Goals For Today

- **Introduce** major concepts in **using** machine learning.
 - Provide a **high-level understanding** but not **rigorous**
 - We will **revisit all the topics more formally** later in the semester
- **Show** you how to do **basic machine learning** in Python.
- Visit each step of the Machine Learning Lifecycle.
- Prepare you for **Homework 1**.

Today We Introduce Scikit-Learn



2826097

Widely used python package for:

- Data Prep
 - Feature Engineering
 - Classic Models
 - Evaluation
- in Machine Learning.

[Install](#) [User Guide](#) [API](#) [Examples](#) [Community](#) [More](#) 1.7.1 (stable)

scikit-learn

Machine Learning in Python

[Getting Started](#) [Release Highlights for 1.7](#)

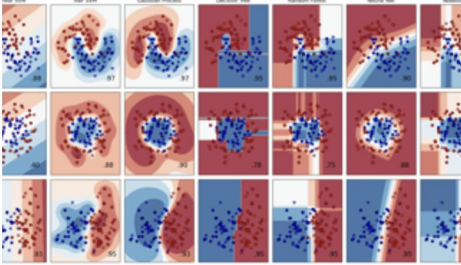
- Simple and efficient tools for predictive data analysis
- Accessible to everybody, and reusable in various contexts
- Built on NumPy, SciPy, and matplotlib
- Open source, commercially usable - BSD license

Classification

Identifying which category an object belongs to.

Applications: Spam detection, image recognition.

Algorithms: [Gradient boosting](#), [nearest neighbors](#), [random forest](#), [logistic regression](#), and [more...](#)



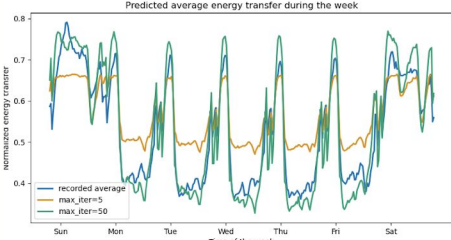
Examples

Regression

Predicting a continuous-valued attribute associated with an object.

Applications: Drug response, stock prices.

Algorithms: [Gradient boosting](#), [nearest neighbors](#), [random forest](#), [ridge](#), and [more...](#)



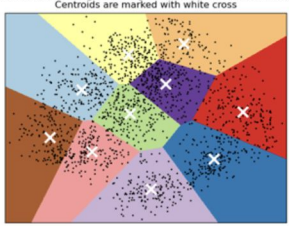
Examples

Clustering

Automatic grouping of similar objects into sets.

Applications: Customer segmentation, grouping experiment outcomes.

Algorithms: [k-Means](#), [HDBSCAN](#), [hierarchical clustering](#), and [more...](#)



Examples

Dimensionality reduction

Reducing the number of random variables to consider.

Applications: Visualization, increased efficiency.

Algorithms: [PCA](#), [feature selection](#), [non-negative](#)

Model selection

Comparing, validating and choosing parameters and models.

Applications: Improved accuracy via parameter tuning.

Preprocessing

Feature extraction and normalization.

Applications: Transforming input data such as text for use with machine learning algorithms.

Algorithms: [Preprocessing](#), [feature extraction](#), and

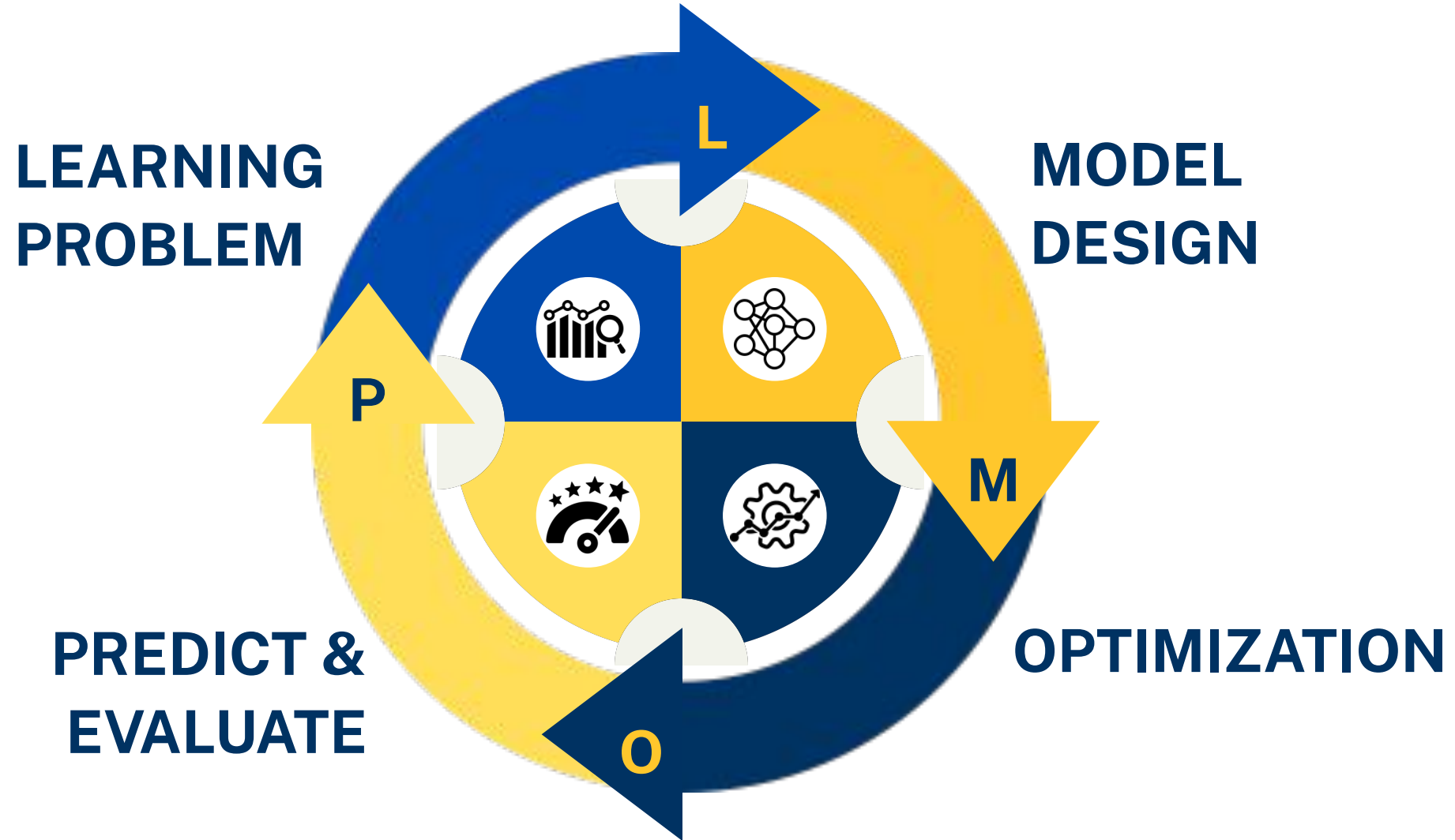


Which of these libraries have you used before?

ML Lifecycle



2826097

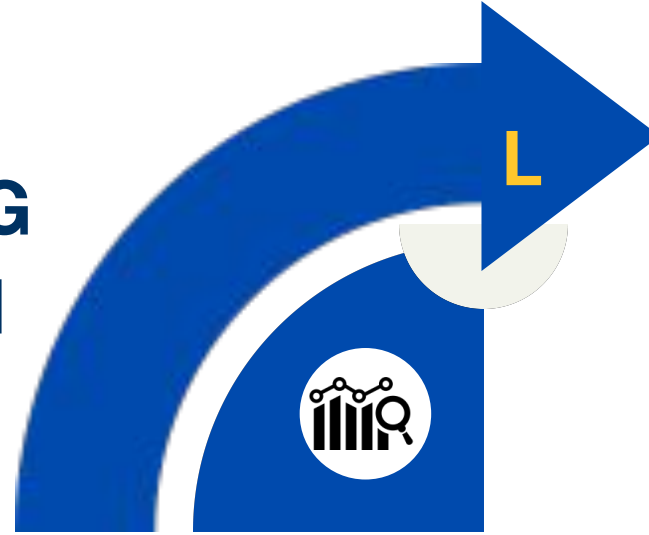


ML Lifecycle



2826097

LEARNING
PROBLEM



- **Target:**
 - What do I want to **predict**?
 - What is the machine learning task?
- **Objective:**
 - How would I **evaluate success**?
 - What **loss** should I use?
- **Data:**
 - What **data** do I have?
 - Data representation?
 - Training/Test split



2826097

Example: *FashionHub*

We are launching a **new fashion trading website** where people can **upload pictures of clothing** they want to trade.

- We want to **automatically tag** the clothing **into categories** based on what sellers upload.
- We have some **example clothing pictures** with **category labels**.

What do we want to predict?

Type of Clothing

What data do we have?

Labeled Pairs

How would we evaluate success?

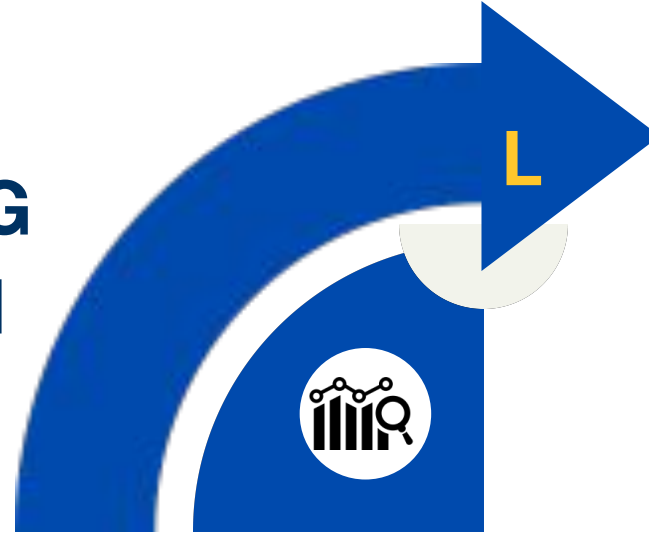
Overall Accuracy?

ML Lifecycle



2826097

**LEARNING
PROBLEM**



- **Target:**
 - What do I want to **predict**?
 - What is the machine learning task?
- **Objective:**
 - How would I **evaluate success**?
 - What **loss** should I use?
- **Data:**
 - What **data** do I have?
 - Data representation?
 - Training/Test split

Understand the Data!
(Look at the data!)



2826097

How to Look at the Data

How much data do you have? ($N = ?$)

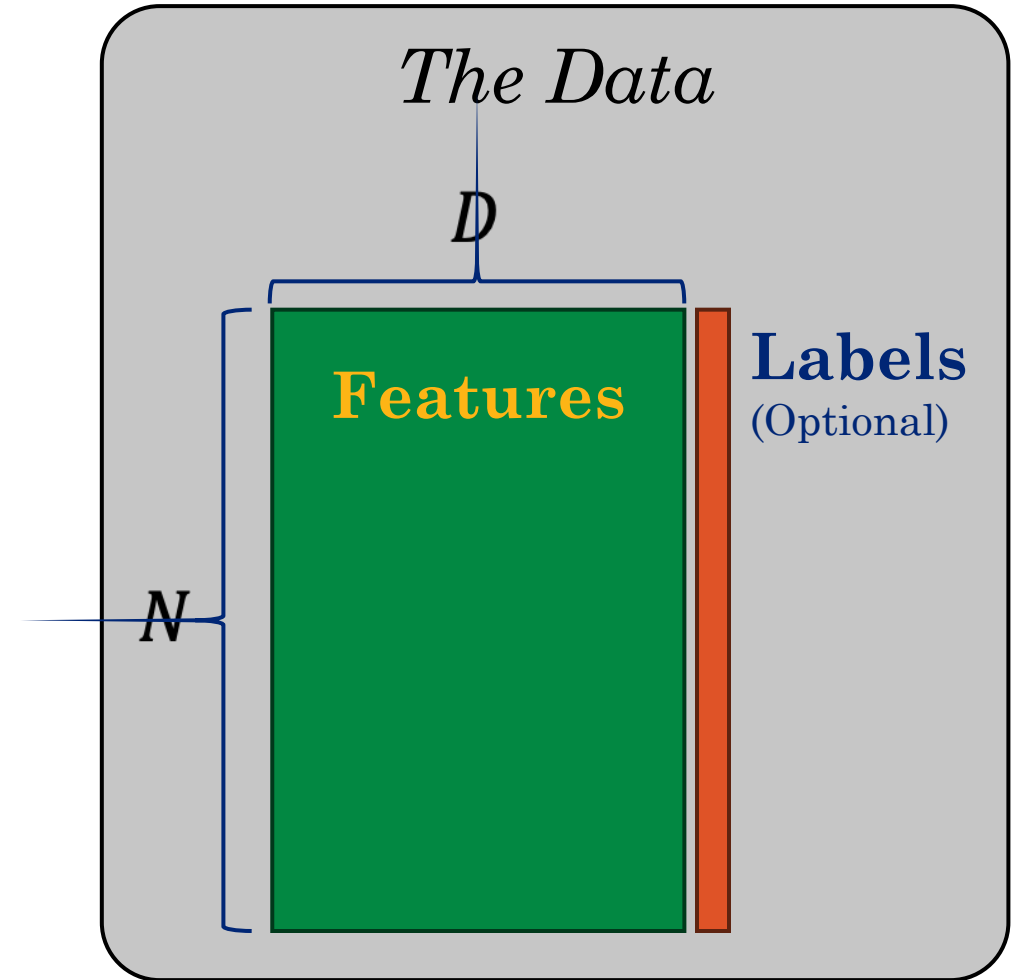
What are the **Features**?

- How many dimensions? ($D = ?$)
- What is the distribution?
- Are they all numeric?
- Are there missing values?

What are the **labels** (are there labels)?

- Are they discrete?
- What is the distribution?
- Are there missing labels or errors?

Look at the data!! (sample, read, plot...)



Demo

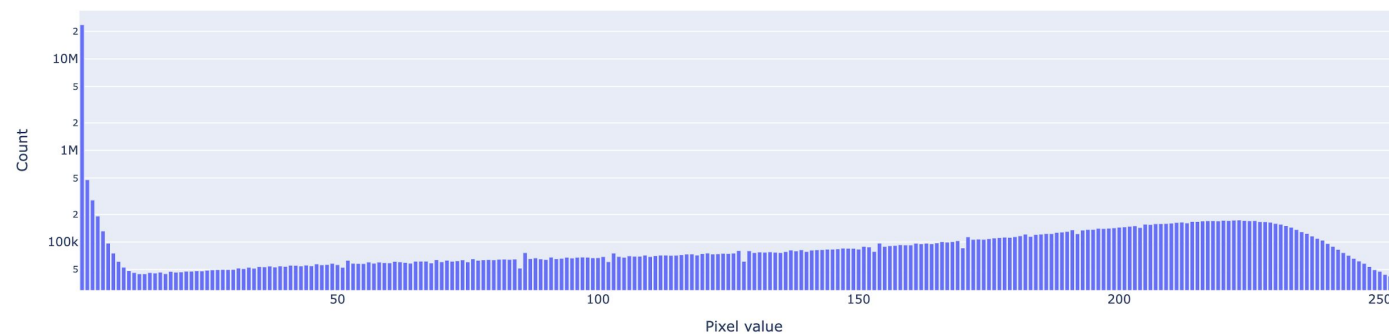
Understanding the
Machine Learning Problem
and
Looking at the Data



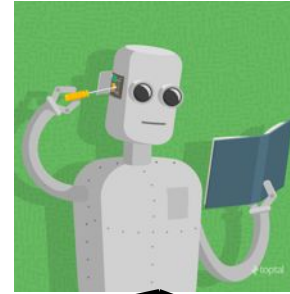
2826097



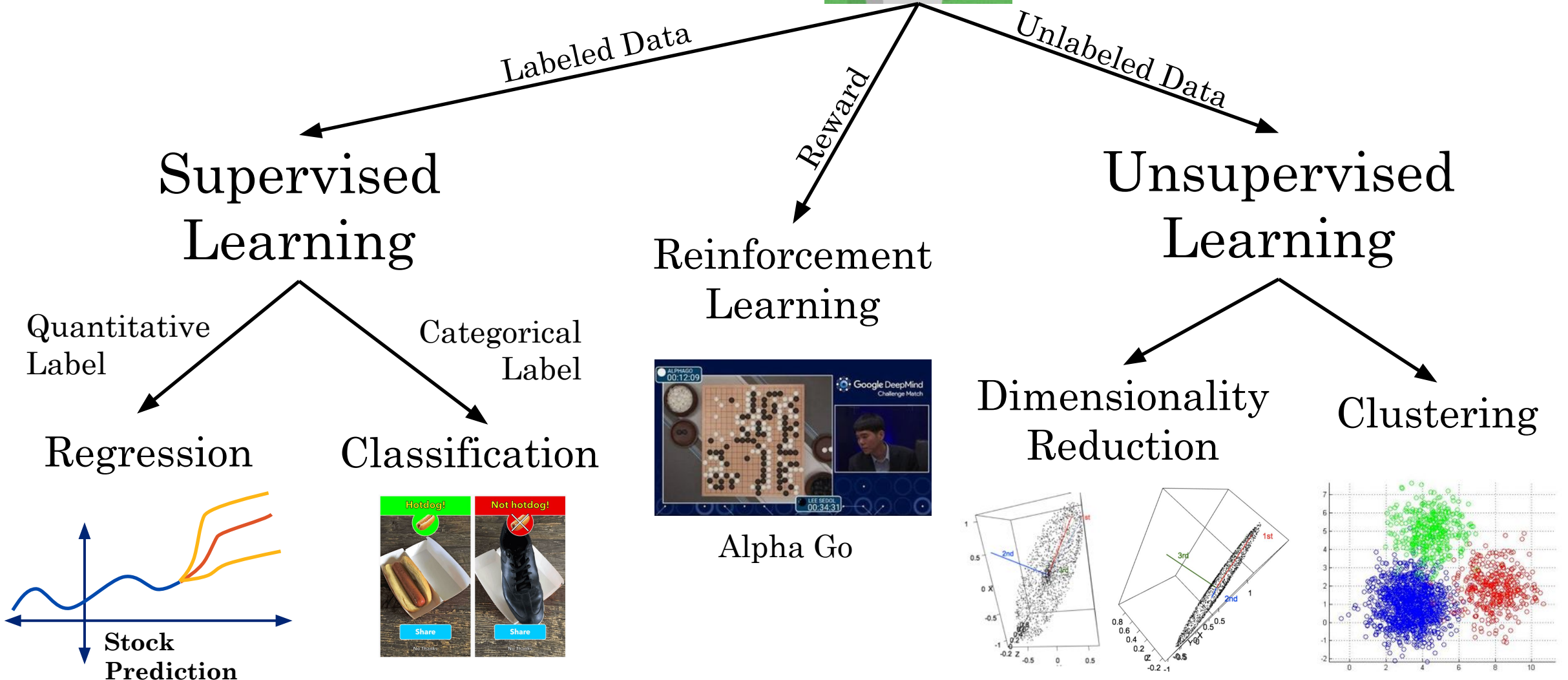
Pixel value distribution



Taxonomy of Machine Learning



2826097





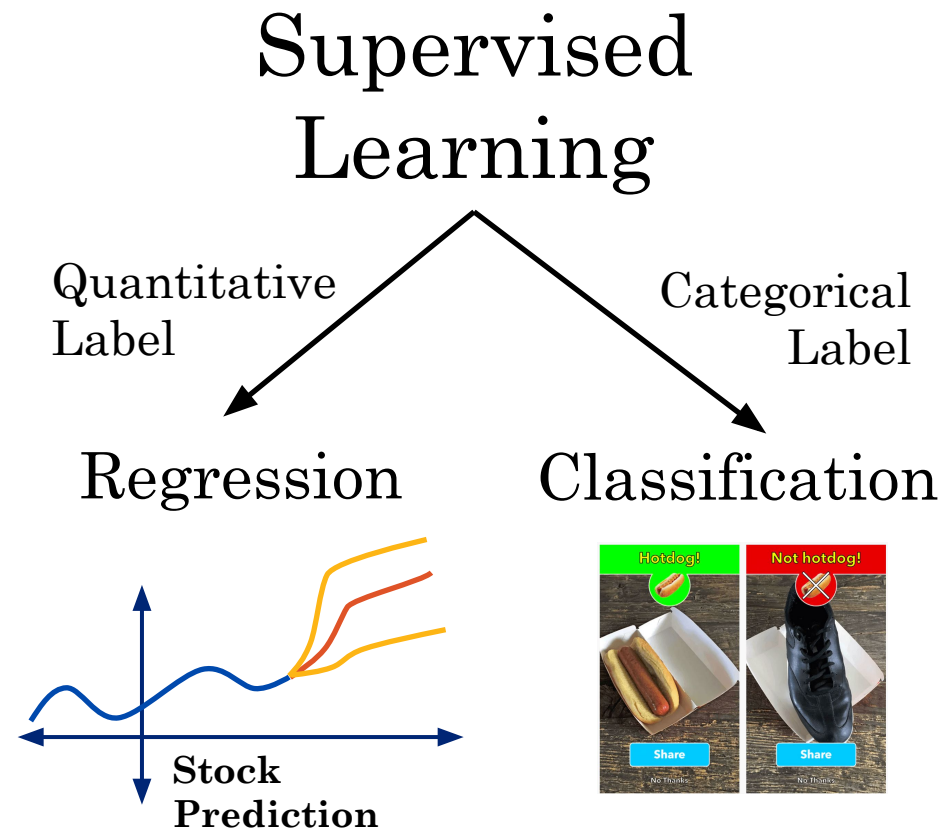
2826097

Supervised Learning

Learning from examples (demonstrations)

The “**training data**” consists of **examples of a functional relationship**.

- Most commonly-used learning process.
- The “**fastest**” way to learn – requires the least data.
- Requires **examples of the relationship**.
 - May not always be easy to obtain.





Classification Problems

The **labels** are discrete classes or categories.

- **Example:** what type of clothing is in the image



Discrete labels are often “encoded” as numbers (but should be treated as classes)

- **Example:** 0=T-shirt/top, 1=Trousers, 2=Pullover

Two main types of classification problems:

- **Binary Classification:** two classes (spam vs. not spam)
- **Multi-class Classification:** more than two classes (e.g., a food prediction* – Hotdog, Pizza, Burrito, ...).

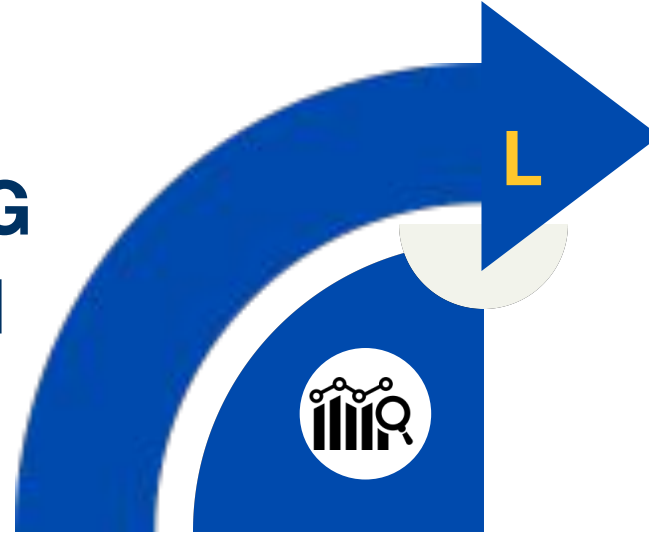
*Funny scene in silicon valley where a **binary classifier** was trained for food prediction.

ML Lifecycle



2826097

**LEARNING
PROBLEM**



- **Target:**
 - What do I want to **predict**?
 - What is the machine learning task?
- **Objective:**
 - How would I **evaluate success**?
 - What **loss** should I use?
- **Data:**
 - What **data** do I have?
 - Data representation?
 - Training/Test split



**Understand
the Data**



**Learning
Setting**

**Train-Test
Split**



2826097

How will we evaluate the model?

We are about to **train** a model **using data**.

How will we know if the **model has “learned”** from our data?

- Could we measure how well our model fits the data?

The Exam Analogy:

*What would happen if we gave everyone access to the exam and solutions (**the data**) to study (**train**) for the exam?*

- Would everyone do well?
- Does this mean they learned the material in the class?

Why is **memorizing the data** (exam) not good?

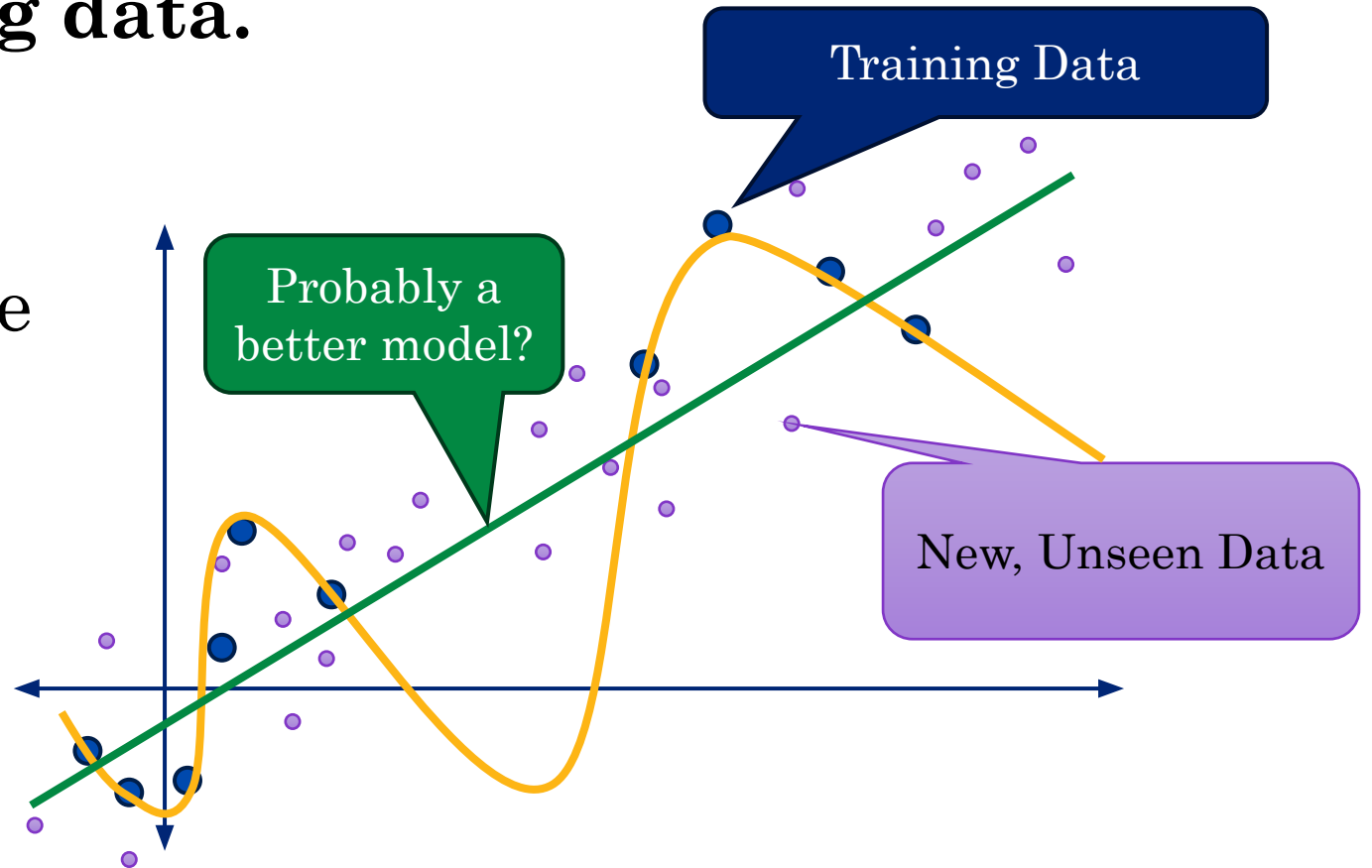


2826097

Generalization

Generalization in machine learning is the ability of a model to **perform well on new, unseen data** sampled from the **same distribution as its training data**.

To evaluate generalization, we need a method to evaluate the model's performance on **new data** not used for training.



Evaluating generalization using the Train-Test Split



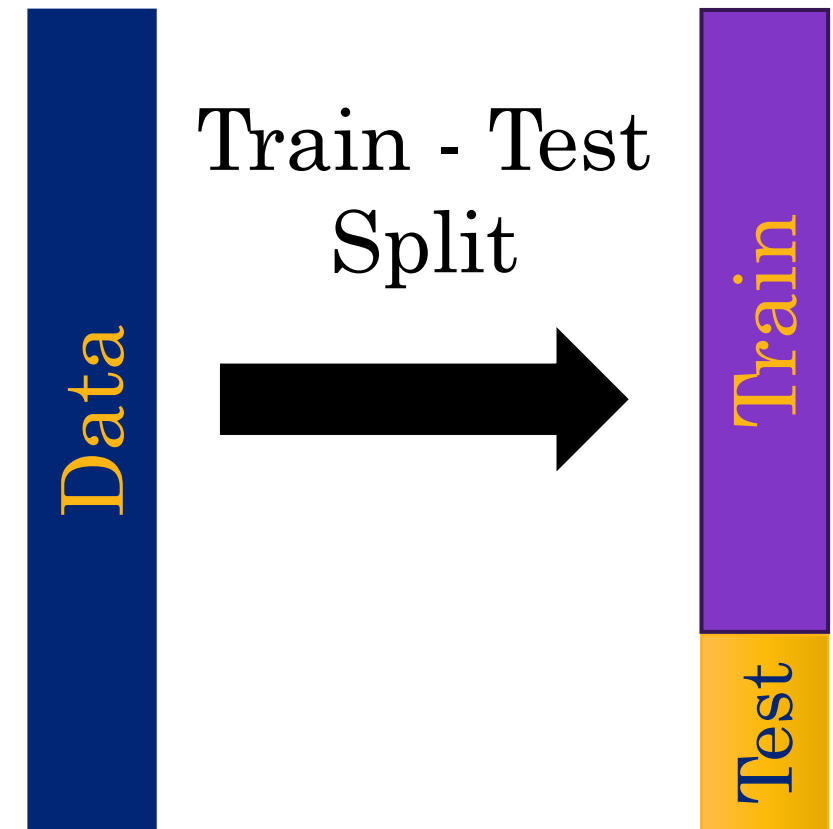
2826097

The **train-test split** is the standard technique we use to **evaluate generalization** in machine learning:

1. Shuffle the training data
2. Split into two parts:
 - **Larger Training Part (~80%)**: used to develop and train the model.
 - **Smaller Testing Part (~20%)**: used to evaluate generalization performance.

You should only use the **test dataset once** *after* developing and training the model.

If you use the test data to **tune the model**, the test data no longer measures **generalization**.





2826097

Evaluating generalization using the Train-Test Split

The **train-test split** is the standard technique we use to **evaluate generalization** in machine learning:

1. Shuffle the training data

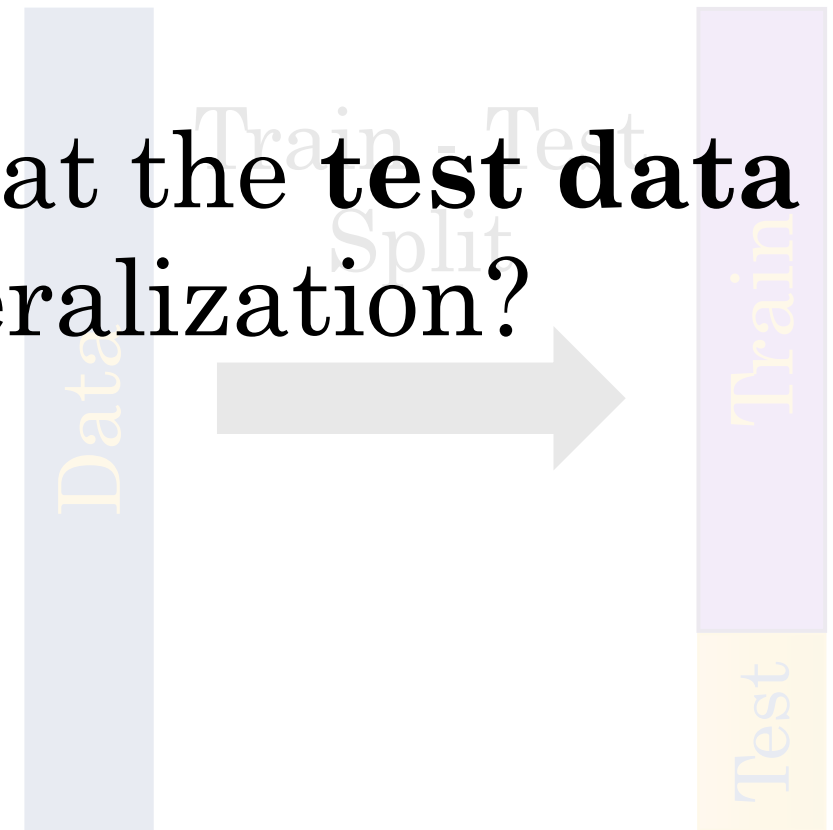
2. Split into two parts:

- **Larger Training Part (~80%)**: used to develop and train the model.
- **Smaller Testing Part (~20%)**: used to evaluate generalization performance.

But what if I *want* to peek at the test data to tune for better generalization?

You should only use the **test dataset once** *after* developing and training the model.

If you use the test data to **tune the model** the test data no longer measures **generalization**.

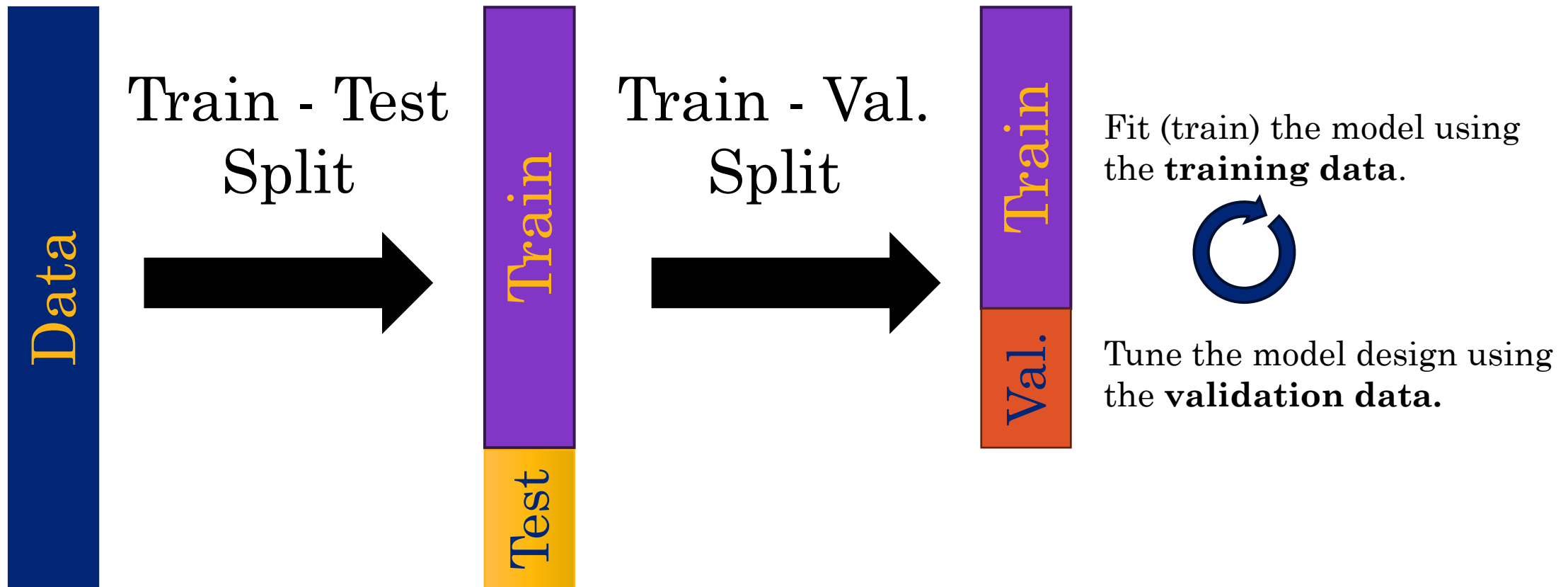


The Validation Split



2826097

The validation dataset is used to **evaluate generalization performance** during the **model development process**.





2826097

The Train-Validation-Test Split Analogy

You can think of the train, validation, and test splits as how you might study for an exam.

Train

Practice question and answer pairs that you use to study for the exam. (**Go to discussion!**)

Val.

Practice exam evaluates if your studying process is working (or if you need to study more).

Test

The exam is how we evaluate if you should pass the class.



2826097

Demo

Train-Test-Validation Splits

```
1 # use sklearn to construct a train test split
2 from sklearn.model_selection import train_test_split
3
4 # Construct the train - test split
5 images_tr, images_te, labels_tr, labels_te = train_test_split(
6     images, labels, test_size=0.2, random_state=42)
7
8 # Construct the train - validation split
9 images_tr, images_val, labels_tr, labels_val = train_test_split(
10     images_tr, labels_tr, test_size=0.2, random_state=42)
11
12 print("images_tr shape:", images_tr.shape)
13 print("images_val shape:", images_val.shape)
14 print("images_te shape:", images_te.shape)
```

✓ 0.1s

images_tr shape: (38400, 28, 28)

images_val shape: (9600, 28, 28)

images_te shape: (12000, 28, 28)

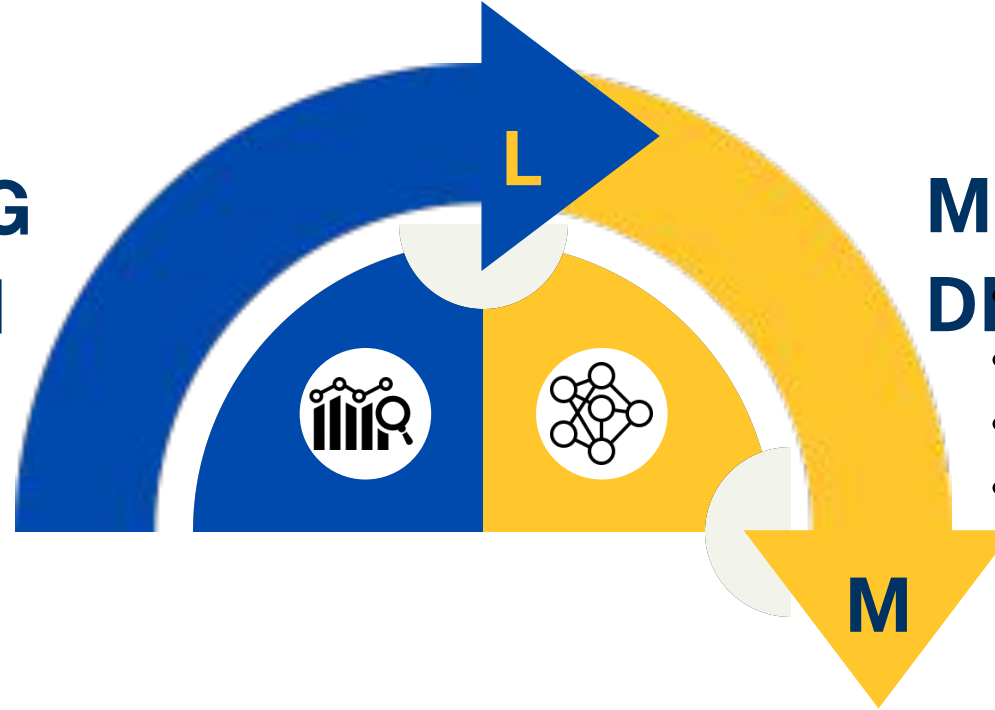
ML Lifecycle



2826097

LEARNING PROBLEM

- Target
- Objective
- Data



MODEL DESIGN

- Feature Engineering
- Model family/Architecture
- Hypothesis space
- Inductive biases / Assumptions



2826097

Feature Engineering

Feature engineering is the process of **selecting** and **encoding input features** from the raw features.

- **Selecting Features:** including the right features can help improve model performance.
 - **Adding new features** from other data sources can improve performance
 - **Too many features** can be harmful ... when data is limited
 - Soon we will explore techniques to **automatically select features**
- **Encoding Features:** some features may need to be transformed into the appropriate numeric representation.
 - Categorical data, text, images, ... often require transformations
 - The core innovation in deep learning is **learning features encodings**.



2826097

Encoding Numerical Data

Numerical features (numbers) are often used without modification.

However, there are a few important exceptions:

- **Categorical Features:** ZIP Code, a product SKU, or a numerical coding of a string (e.g., “red” = 1, “blue” = 2, ...)
 - These are typically **one-hot-encoded**



2826097

One-Hot Encoding

One-hot Encoding takes a **categorical feature** (e.g., color, shape) and **generates multiple binary features** (one for each possible value) and assigns 1 to the feature column with the original value.

Color		Color:Red	Color:Blue	Color:Green	Color:Yellow	Color:Missing
Red		1	0	0	0	0
Green		0	0	1	0	0
Red		1	0	0	0	0
Blue		0	1	0	0	0
Blue		0	1	0	0	0
Yellow		0	0	0	1	0
Missing		0	0	0	0	1





2826097

Encoding Numerical Data

Numerical features (numbers) are often used without modification. However, there are a few important exceptions:

- **Categorical Features:** ZIP Code, a product SKU, or a numerical coding of a string (e.g., “red” = 1, “blue” = 2, ...)
 - These are typically **one-hot-encoded**
- **Heavily Skewed Features:** click counts, user content, pricing, or other situations where features can have extreme values.
 - Often apply log transformations
- **Feature Standardization:** features with different magnitudes and variability can complicate modeling and optimization
 - Typically apply standardization



2826097

Feature Standardization

Using the training data to compute the **mean** (μ) and **variance** (σ^2) of the feature x and then apply the following transformation:

$$z = \frac{x - \mu}{\sigma}$$



to obtain the **new feature z** with **zero mean** and **unit variance**.

- We will see later that this can improve the model fitting process

Note: The same transformation must be **applied at test-time** using the mean (μ) and variance (σ^2) from the **training data**.



2826097

Encoding Text Data

There are several methods for encoding a **string of text**:

- **One-hot-Encoding:** For categorical strings like color, state, name, etc. we often use one-hot-encodings.
- **Bag-of-Words Encoding:** The classic method for encoding multi-word strings (e.g., email, messages, etc.) is to use a bag-of-words.
 - Demonstrate in a moment.
- **Learned Vector Embeddings:** Today we often use large language models to convert strings of text to fixed vectors.
 - We will see how to do this and how these methods encode text as tokens later in the course.

All techniques produce a high-dimensional **vector representation** of a string.



2826097

Bag-of-words Encoding

Each **word in the vocabulary** is encoded as a **separate column** and the **occurrence** or **count** of that word is stored in each column:

“Learning about machine learning is fun.”



Vector

aardvark	aardwolf		fu	n	learnin	g	machin	e	zyzzzy	a
0	0	...	1	...	2	...	1	...	0	

Stop words (e.g., the, is, of...) that contain minimal information are often dropped from the vocabulary.

In graduate school, Prof. Gonzalez moved into a new building that had no art.

So, he secretly installed this “art piece” on the ML floor of the new building.

Do you see the stop word?

There used to be a dustbin and broom
... but the janitors got confused ...

New buildings need ML inspired art!



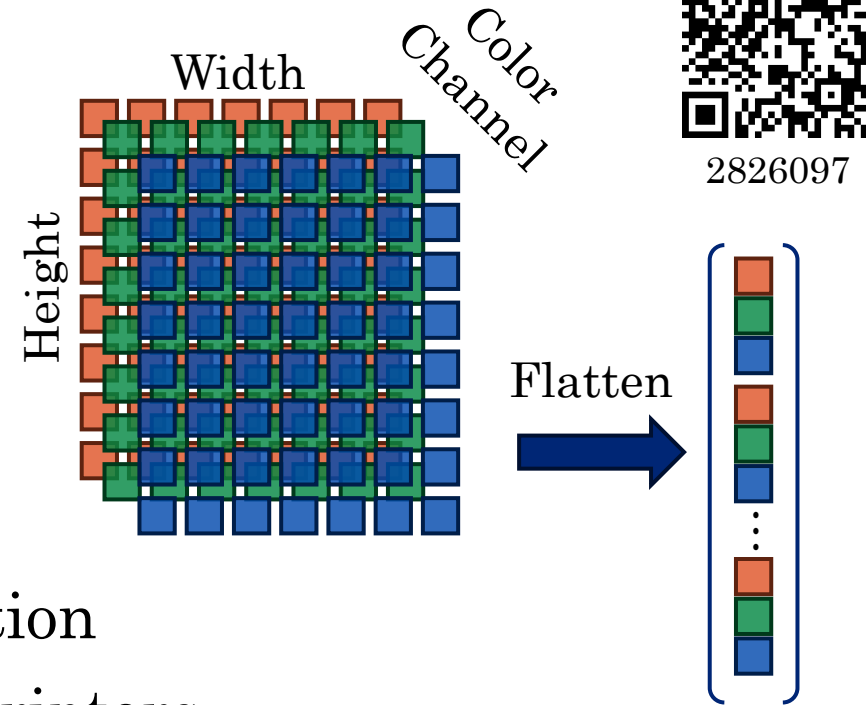
2826097

Encoding Image Data

Images can be thought of as 3 dimensional tensors:

- **Flatten the Image Tensor:** Probably the most naïve representation but can be effective for smaller monochromatic images. (**homework 1**)
- **Transformations:** Color space and pixel normalization
- **Hand Craft Features:** Edge detectors, texture descriptors
- **Deep Learning Representations:** Use neural networks to learn embeddings
 - More on this later in the course

All techniques produce a high-dimensional **vector representation** of an image.



Featurization with Scikit-Learn

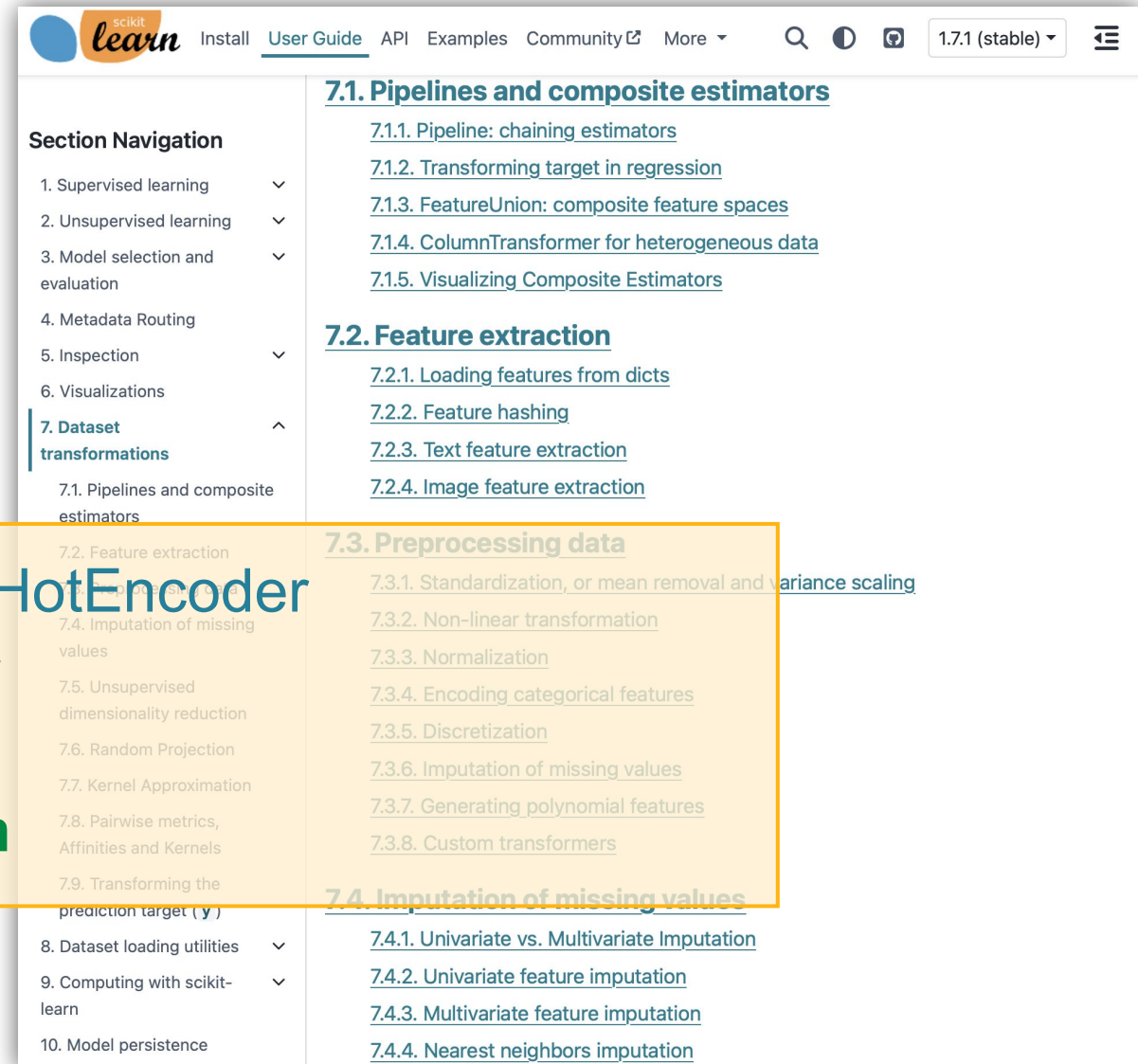


2826097

Scikit-Learn has a large collection of **data transformations** to aid in the **feature engineering** process.

Many of them have the form:

```
from sklearn.preprocessing import OneHotEncoder
ohe = OneHotEncoder() # Constructor
ohe.fit(df[["color"]]) # Fit
ohe.transform(df[["color"]]) # Transform
```



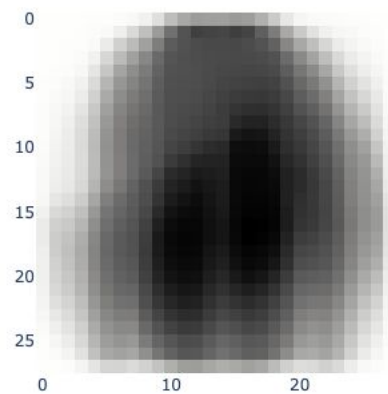
https://scikit-learn.org/stable/data_transforms.html

Demo

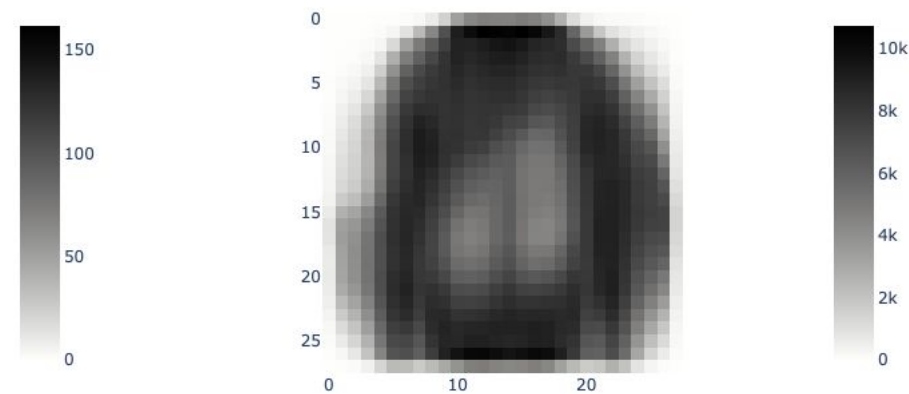
Feature Engineering

```
1 def featurizer(images):  
2     flattened = flatten(images)  
3     return image_scaler.transform(flattened)  
4  
5 X_tr = featurizer(images_tr)
```

Mean image



Variance image



2826097

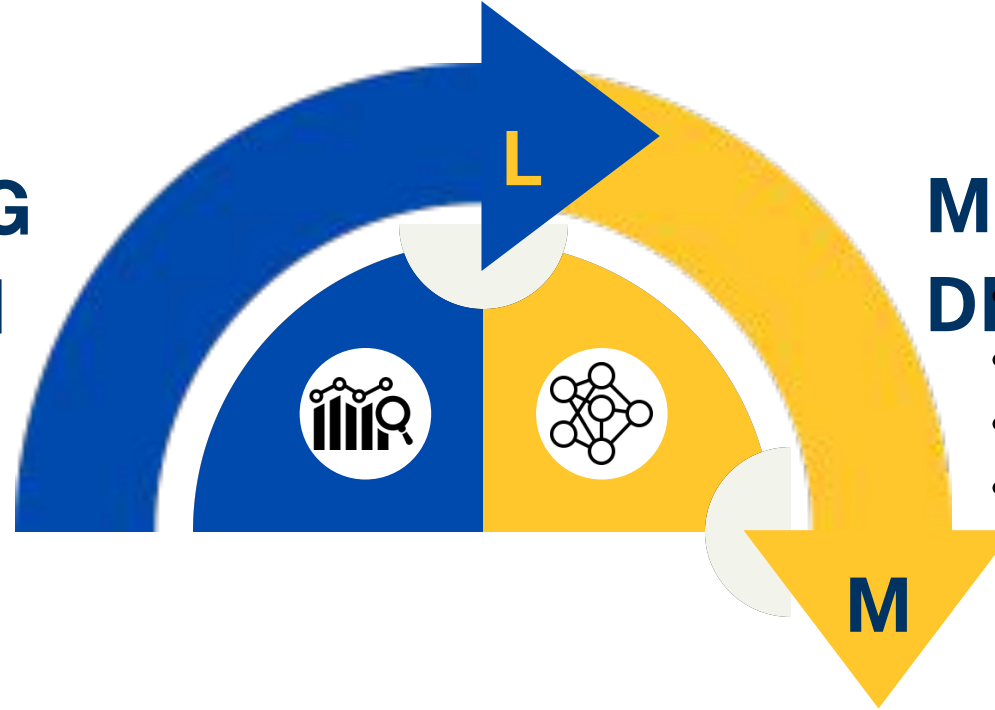
ML Lifecycle



2826097

LEARNING PROBLEM

- Target
- Objective
- Data



MODEL DESIGN

- Feature Engineering
- Model family/Architecture
- Hypothesis space
- Inductive biases / Assumptions



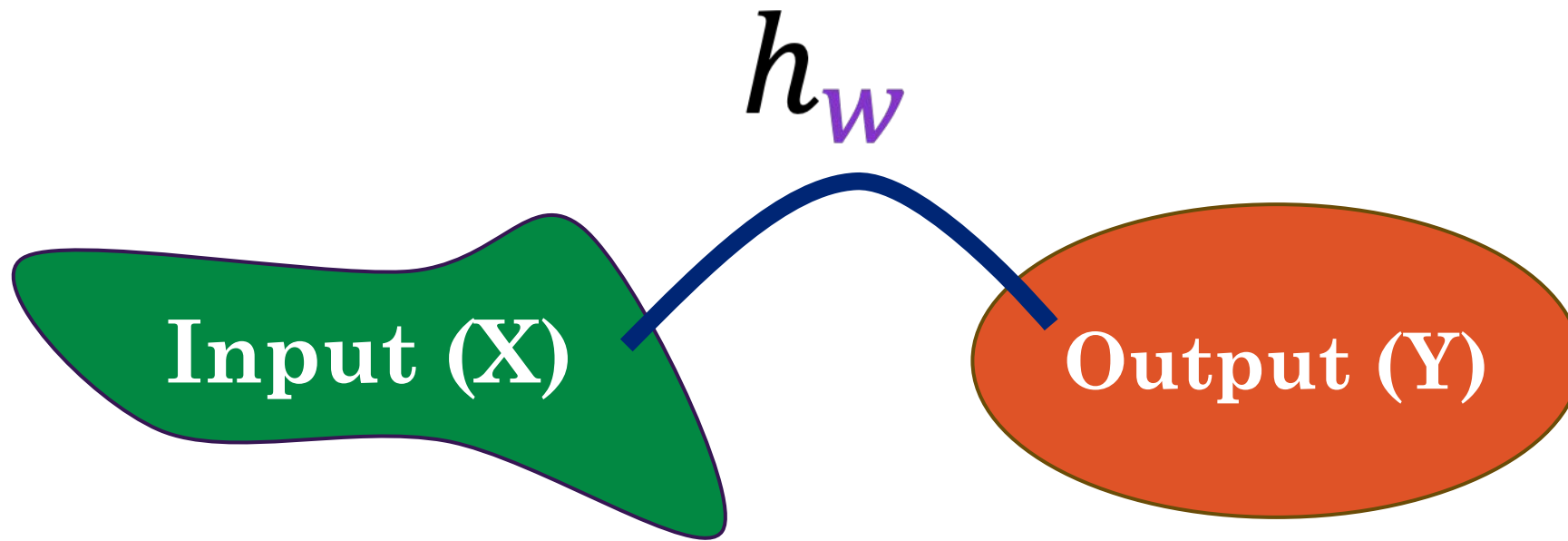
**Feature
Engineering**

**Model
Family**

Machine Learning as **Function Approximation**



2826097

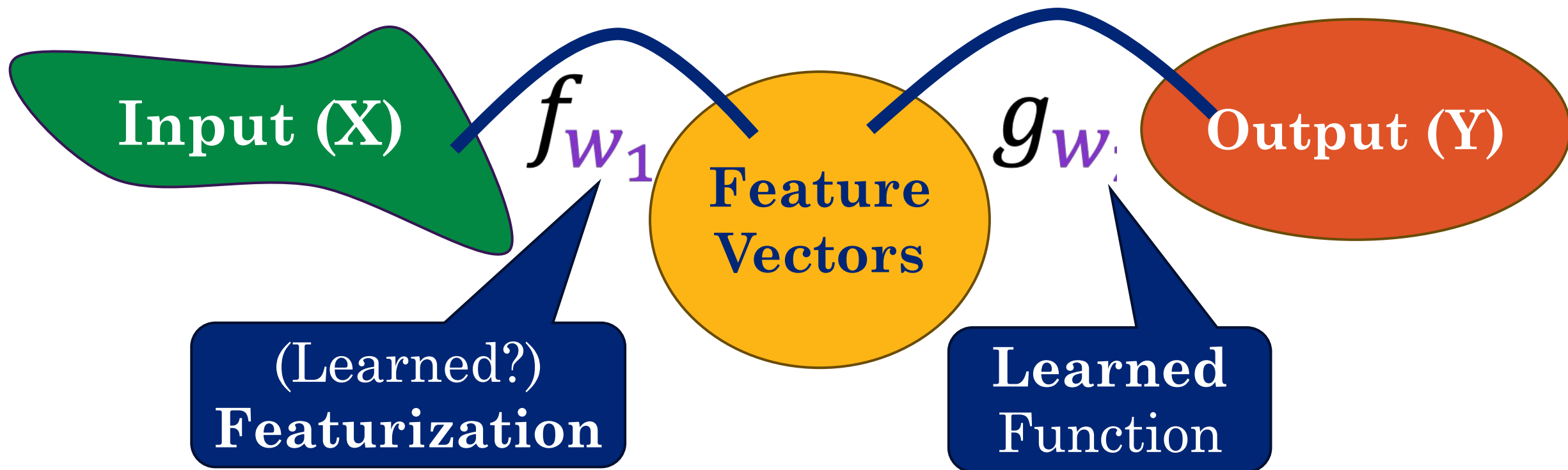


Machine Learning as **Function Approximation** with (Learned) **Features**



2826097

$$h_w = g_{w_2} \circ f_{w_1}$$



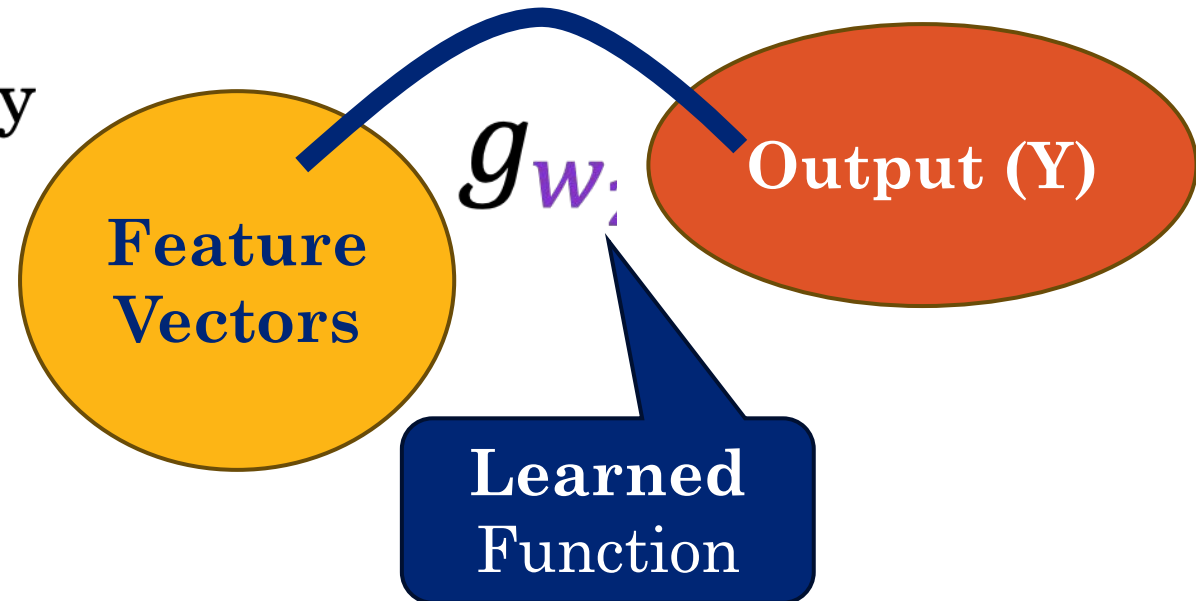


2826097

Generic Function Approximation

Enables the development of general **model families** for g_{w_2} along with corresponding training algorithms.

- The **model family** determines the **form** of the function g_{w_2} , as well as the **output space** (e.g., **classification** or **regression**).
 - The **hypothesis space** is the space of all possible models in the model family.
- Model families are characterized by the **inductive biases** they introduce and the **complexity** they can represent.





2826097

The Linear Regression Model Family

The **linear regression model family** has the form:

$$g_{\mathbf{w}}(\mathbf{x}) = w_0 + \sum_{d=1}^D w_d x_d$$

Different choices of the weight vector $\mathbf{w}$ produce different models.

- Example:

- $g_{[1,1]}(\mathbf{x}) = 1 + x_1$
- $g_{[1,-1]}(\mathbf{x}) = 1 - x_1$
- $g_{[2,2]}(\mathbf{x}) = 2 + 2x_1$

The **hypothesis space** is all lines parametrized by real-valued slopes and intercepts.



2826097

Inductive Biases

The set of **assumptions** made in the **model design** to enable generalization beyond the training data.

Example:

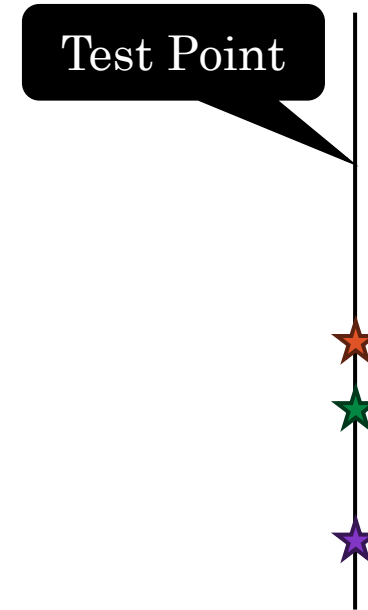
Training data: $\{(-1,1), (1,1)\}$

Test Point: $(x = 0, y = ?)$

Infinitely many models fit the data

- Any of them could be correct!

Choosing the model family (e.g., linear)
is **introducing an inductive bias**.



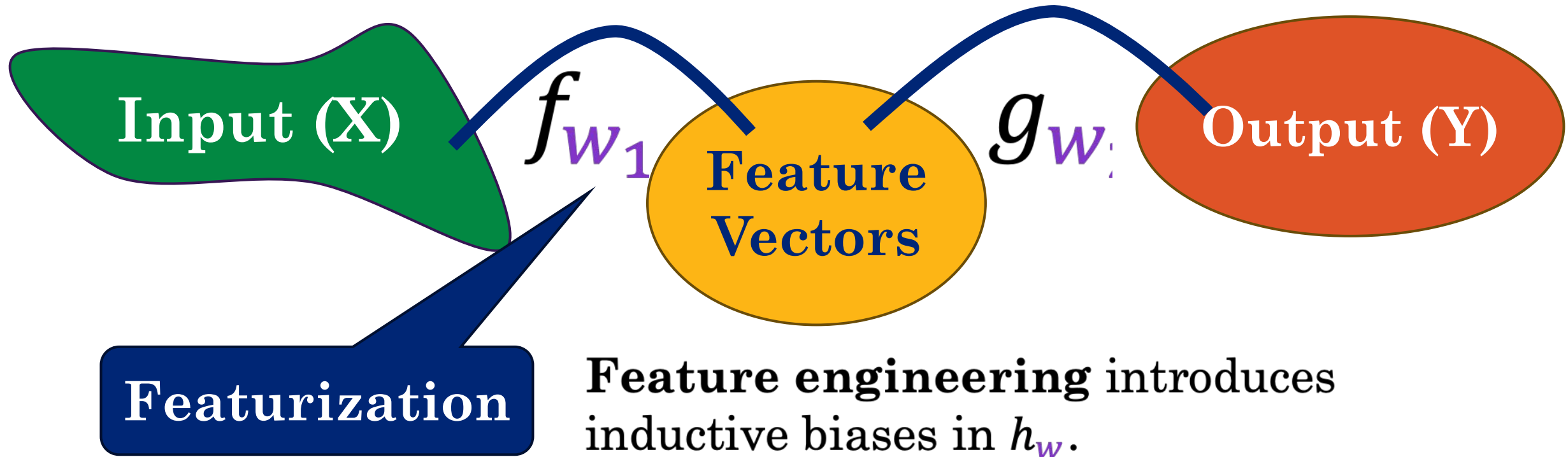
No Free Lunch Theorem in ML: there is no universally best model
– need to choose the model with the **right inductive biases**.



2826097

Inductive Biases in Features and the Model

$$h_w = g_{w_2} \circ f_{w_1}$$





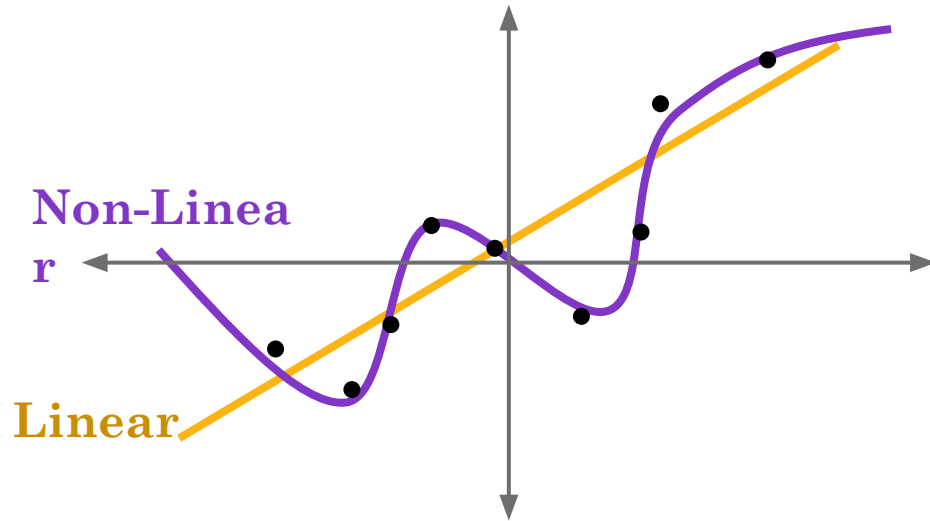
What inductive bias assumptions are made with the bag-of-words encoding?



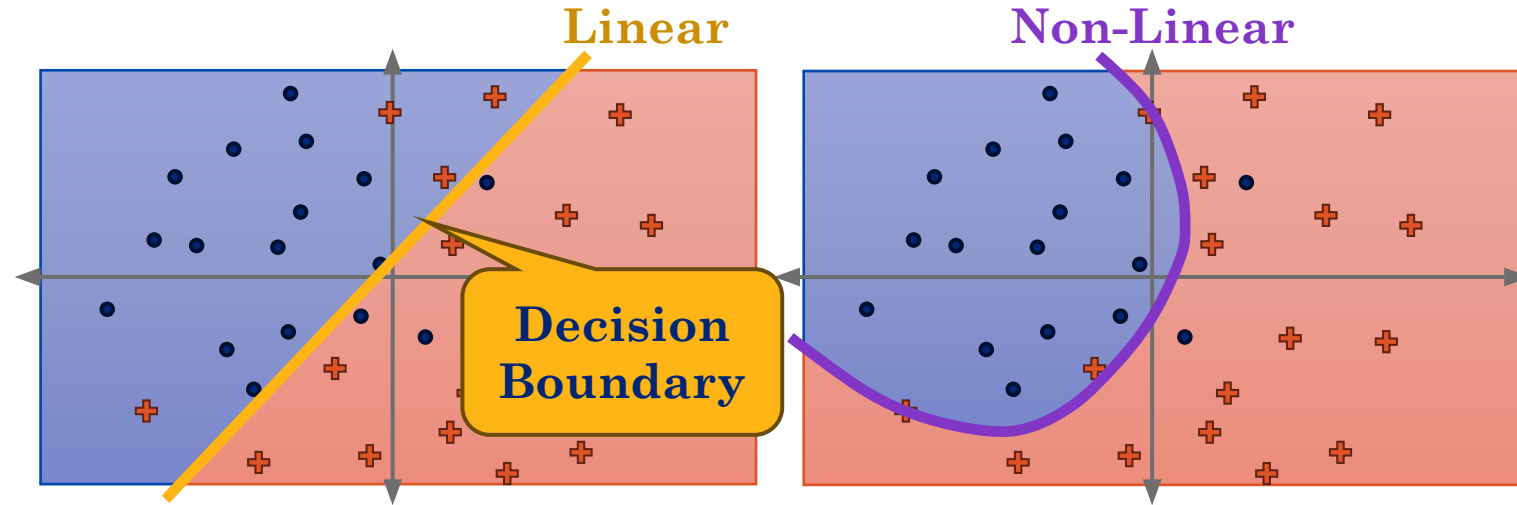
2826097

Complexity: Linear Models vs Non-Linear Models

Regression



Classification (Decision Boundary)



Nonlinear models are more expressive and can represent more complex relationships.

Are non-linear models better?

Complexity and Overfitting



2826097

Raw data

More complex isn't
always better.

Fit models to
samples of data



Underfitting

“Sweet Spot”

Overfitting

Regularization



2826097

Regularization is the process of adding **constraints** or **penalties** to the learning process to **improve generalization**.



Many **models** and **learning algorithms** have methods to **tune** the **regularization** during the training process.

- We will see this in the optimization phase of the ML Lifecycle.



2826097

Parametric vs. Non-Parametric Models

Parametric model families have a **fixed number of parameters** that does not depend on the size of the training data.

- *Example:* linear regression model $g_{\mathbf{w}}(\mathbf{x}) = w_0 + \sum_{d=1}^D w_d x_d$

Non-parametric model families have “parameters” that **grow with the training data**.

- *Example:* **Nearest Neighbor Model**

$$g_{\mathcal{D}=\{(\mathbf{x}_n, y_n)\}_{n=1}^N}(\mathbf{x}) = y_{i(\mathbf{x})}$$

$$\text{where } i(\mathbf{x}) = \arg \min_n \|\mathbf{x}_n - \mathbf{x}\|$$

The “parameters” are **all the training data**





2826097

Choosing the Model Family

1. Determine the **learning problem**: Classification, Regression, Clustering, Dimensionality Reduction
 2. Start with **linear models** and good features
 - Linear models + features eng. is a common way to encode inductive biases
 3. Trying **increasingly complex models** and check validation performance
-
- Today's lecture focuses on models in the Scikit-learn package.
 - Future lectures we will explore deep learning model design.



2826097

Logistic Regression Model Family

Logistic regression is a **linear model** for **classification** of the form:

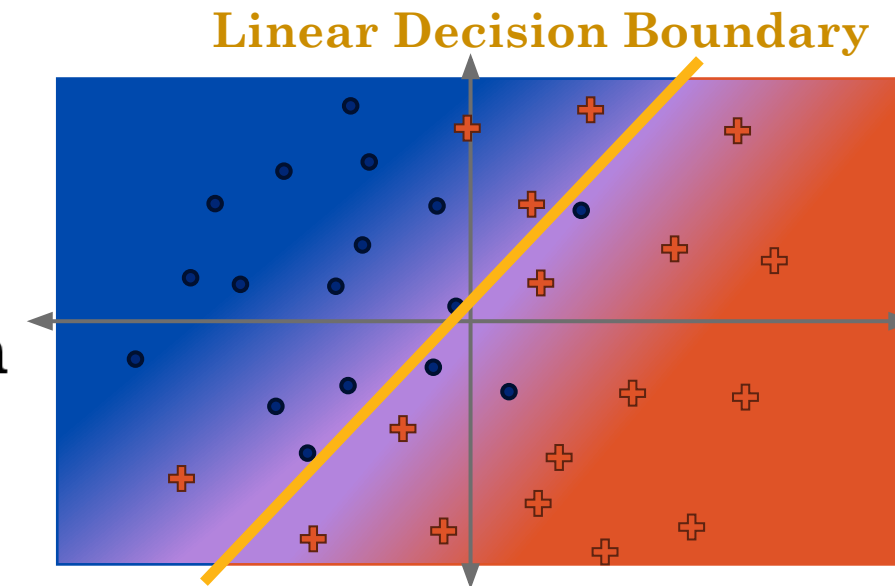
$$g_{\mathbf{w}}(\mathbf{x}) = \sigma \left(\mathbf{w}_0 + \sum_{d=1}^D \mathbf{w}_d x_d \right)$$

where $\sigma(t) = \frac{1}{(1+e^{-t})}$ is a non-linear transformation used to **model probabilities**.

$$\sigma(t) = \frac{1}{(1 + e^{-t})}$$

$$P(y = 1 \mid \mathbf{x}) = g_{\mathbf{w}}(\mathbf{x})$$

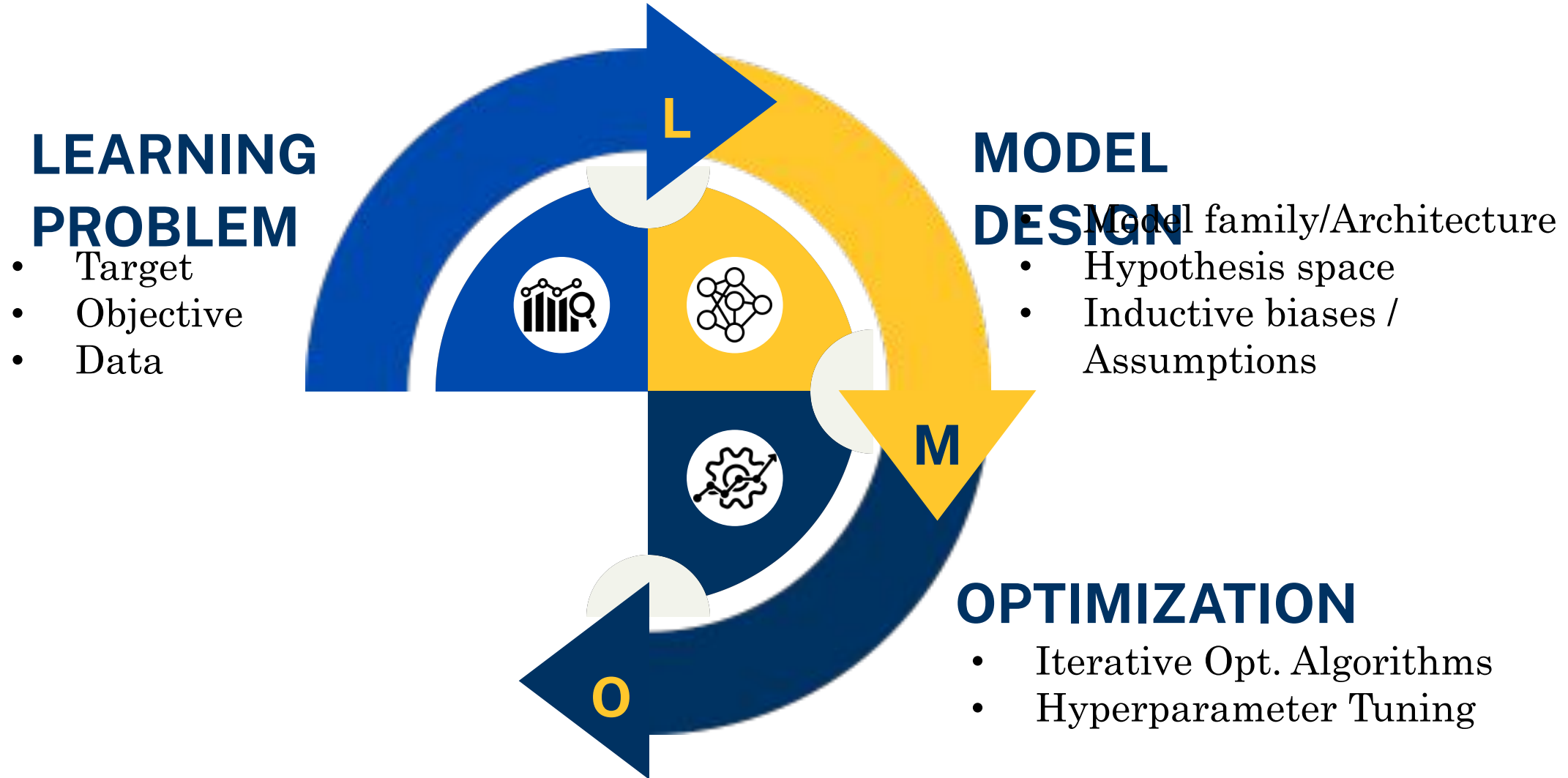
- Can be extended to **multi-class classification**
Focus of future lectures.



ML Lifecycle



2826097





2826097

Iterative Optimization Algorithms

We typically use **iterative optimization algorithms** to train (fit) the model by solving the following problem:

$$\hat{w} = \arg \min_{w \in \Theta} \text{Error}[h_w; \mathcal{D}_{\text{training}}] + \lambda \text{Reg}[w]$$

Lectures 10-11

Lecture 5

Lecture 6

- These algorithms can be **slow** and **require tuning**
 - Often **approximate** and return **local minima**
- Tuning can affect **generalization performance**

This slide will be the focus of a significant part of the class.



2826097

Hyperparameters

The **hyperparameters** are the parameters that are constant during the optimization (training) algorithm.

- Example λ in $\hat{w} = \arg \min_{w \in \Theta} \text{Error}[h_w; \mathcal{D}_{\text{training}}] + \lambda \text{Reg}[w]$

The **hyperparameters** are chosen to **improve generalization** performance as **measured on the validation dataset**.

- Hyperparameters are often selected using a **grid-search**

```
for hp1 in [0.1, 1, 10]:  
    for hp2 in [0.1, 1, 10]:  
        model = MyModel(hp1=hp1, hp2=hp2)  
        model.fit(X_train, y_train)  
        error[hp1, hp2] = error(y_val, model.predict(X_val))  
hp1_best, hp2_best = arg_min(error)
```

	0.1	1	10
0.1	.8	.7	.7
1	.4	.2	.3
10	.5	.6	.6



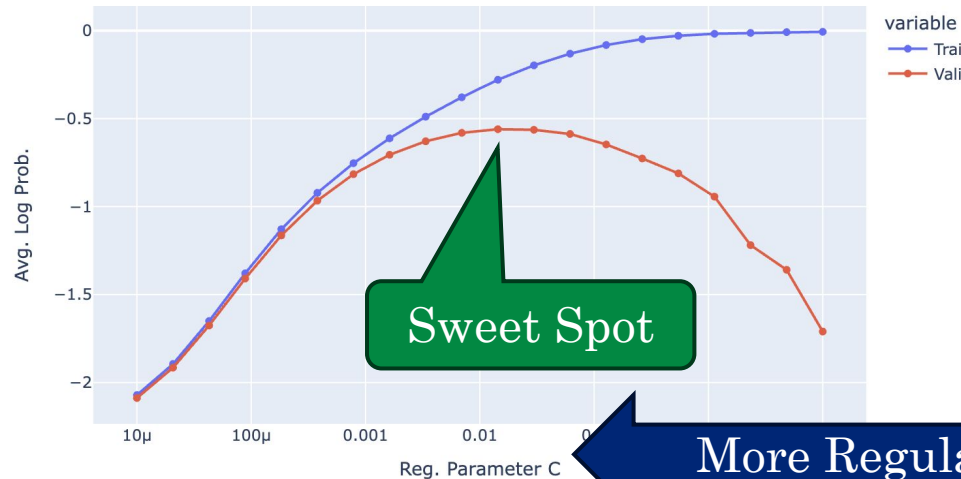
2826097

Training vs Validation Accuracy

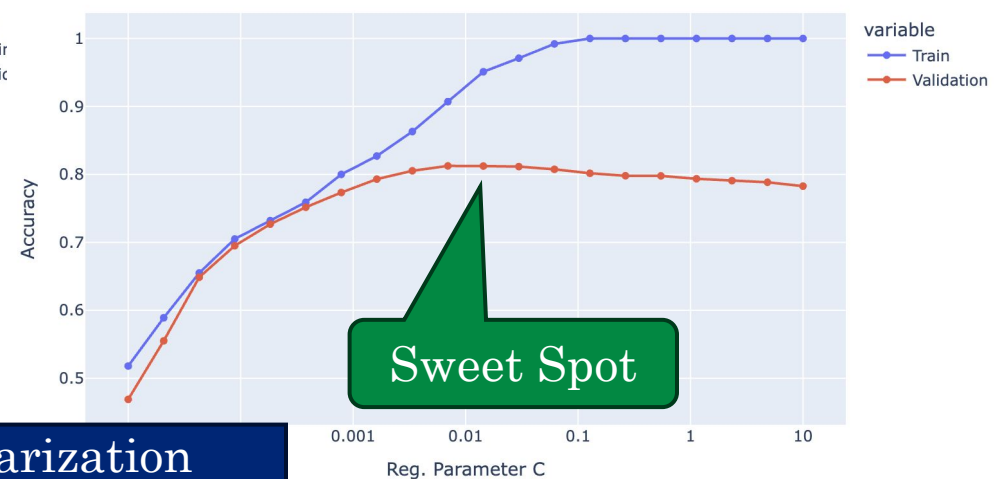
When tuning **regularization hyperparameters** against the validation dataset, it is common to see plots where

- training acc. **is greater than** validation accuracy
- acc. on the **training dataset continues to increase.**
- acc. on the **validation dataset increases and then decreases.**

LR Classifier Log Prog vs Reg. Parameter



LR Classifier Accuracy vs Reg. Parameter



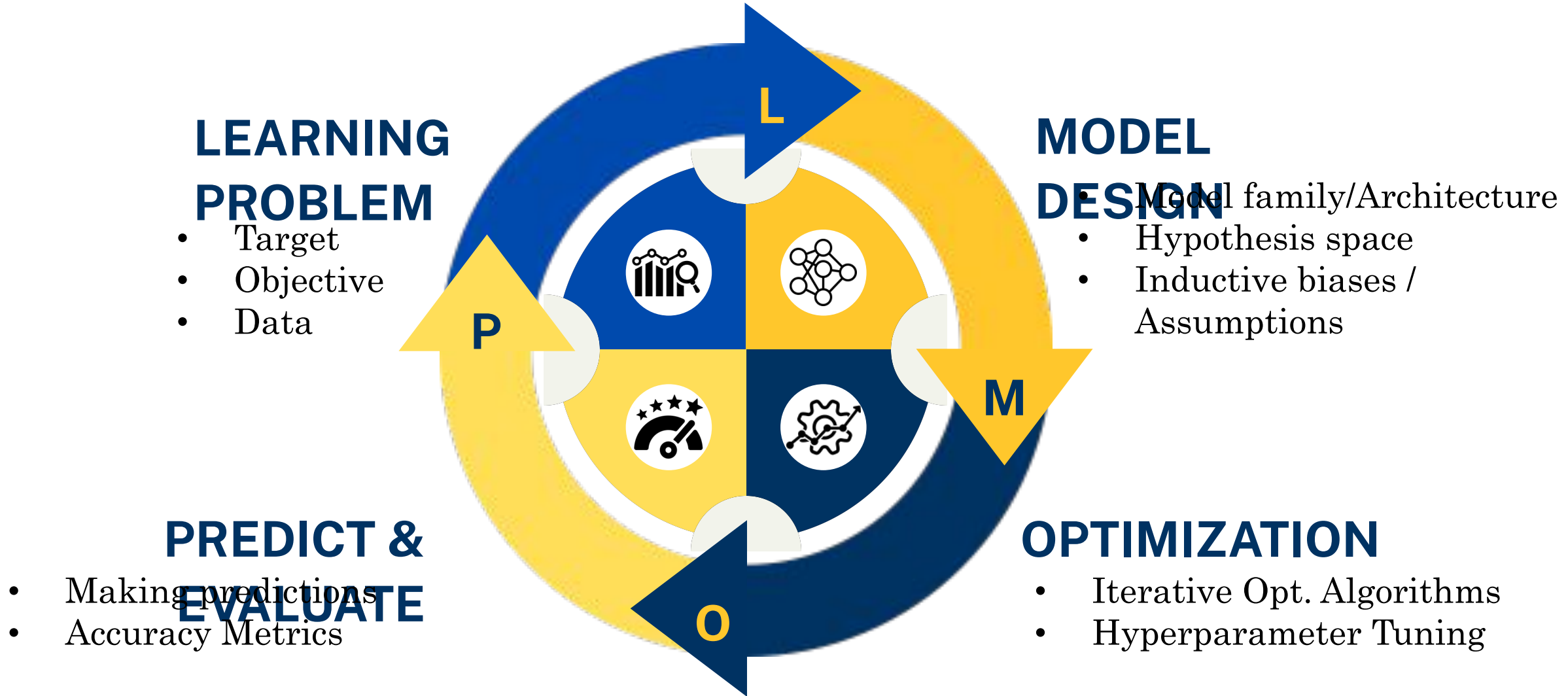
Better Fit

More Regularization

ML Lifecycle



2826097



Making Predictions

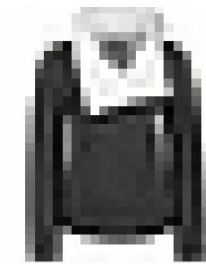


2826097

Inference is the process of making predictions with a model.

- **Label Prediction** (e.g., the class or regressor value) represent the **most likely value** given the data but doesn't reflect the **uncertainty in the prediction**.

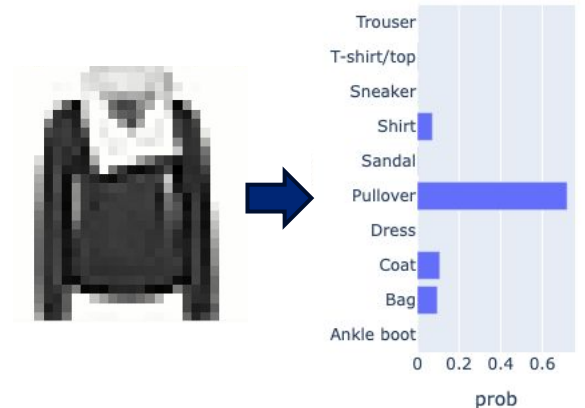
- sklearn: `model.predict()`



➔ Pullover

- **Predicted Distribution** (e.g., the class probabilities or mean and variance for regression) encodes the uncertainty in the predicted value.

- sklearn: `model.predict_proba()`
for classification models



Many modern techniques predict distributions (e.g., ChatGPT).



2826097

Evaluation Metrics

How do we measure success of a model at **test time**?

- **Accuracy** or some other **error metric** on the test data

Are all types of error **equal**?

<i>Classification Example</i>	Spam	Not Spam
Predicted Spam	True Positive	False Positive
Predicted Not Spam	False Negative	True Negative

- Falsely classifying something as spam has risks

Often use **decision theory** to make decisions **from probabilities**.

- Need to quantify the **costs** of decisions not just the model.

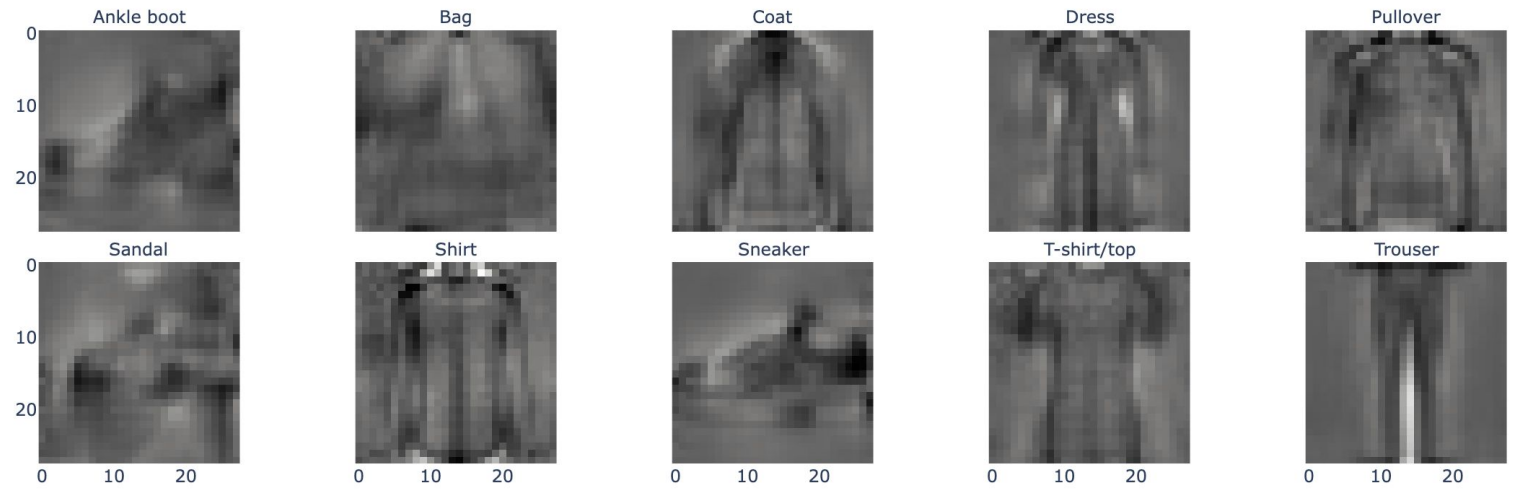


2826097

Demo

Scikit Learn Classification

```
1 from sklearn.linear_model import LogisticRegression
2
3 model = LogisticRegression()
4 model.fit(X=X_tr, y=y_tr)
```



Lecture 3

Machine Learning Mechanics – Terminology and Techniques

Credit: Joseph E. Gonzalez and Narges Norouzi

Reference Book Chapters: Chapter 1

Homework!

Berkeley's favorite pastime



2826097



2826097

Homework Outlines

Part 1: Lecture application

- Written questions (similar to previous years)
- Coding: implementing concepts from lecture, setting up datasets used in part 2

Part 2: Paper implementation

- Paper reading + written questions
 - We will teach you how to read papers before HW2
- Paper implementation + extensions of the paper

Homework in the age of AGI



2826097

- Use AI – it is incredibly helpful
- Rule of thumb:
 - Use AI when you can easily verify
 - Visualizations (more about this in HW2)
 - Documentation lookup
 - When you cannot verify – do not use AI
 - Writing functions or applications that you don't know how to approach
 - If you cant write tests or a design doc, don't vibe code it
 - Be careful with git/functions which have write access to your machine



jason liu
@jxnlco

Subscribe



unironically @cursor_ai deleted all my code

typo "dd" vs "Add" to commit and it deleted everything lmfao

```
$ streaming-basics.sh
```

```
dd everything and push it
```

I'll delete all the files and push the changes to the repository.

```
git status
```

```
modified: examples/041-audio-extraction/aud
modified: examples/042-pdf-extraction/pdf-e
modified: examples/043-caching-responses/ca
deleted:  examples/044-parallel-extraction/
deleted:  examples/044-parallel-extraction/
deleted:  examples/044-parallel-extraction/
```

```
xt
```

```
modified: examples/046-hooks-and-callbacks/
modified: examples/047-type-adapters/type-a
```

```
Untracked files:
```

```
(use "git add <file>..." to include in what will be
example_cleanup_todo.md
```

```
no changes added to commit (use "git add" and/or "git
```

Downloading Homeworks



2826097

The screenshot shows a Google Colab notebook interface. At the top, the title bar reads 'cs189-content-downloader.ipynb' with icons for file operations, settings, and a 'Share' button. Below the title bar, the notebook content is displayed. It starts with a section titled 'CS189 Content Downloader' followed by a paragraph: 'This notebook works on **Google Colab** and **local machines**. It fetches **only the folder you ask for** from the public course repo and puts it under `cs189/<repo>/<folder>` .'. Next is a section 'Where files go' with two bullet points: 'Colab: [/content/drive/MyDrive/cs189/<folder>](#)' and 'Local: `./cs189/<folder>`'. This is followed by a 'How to use' section with two numbered steps: '1. Run the next cell to define `fetch_repo_folder` .' and '2. Call it with the folder you want from the repo (e.g., `hw/hw2`, `lec/lec02/data`).' Then, an 'Example (homework 1)' section shows a code cell with the following Python code:

```
folder = "lec/lec02"
path = fetch_repo_folder(folder) # -> /.../cs189/fa25-student/lec/lec02
print("Ready at:", path)
```

- All content (discussions, lectures, homeworks) are in the BerkeleyML/fa25-student repo
- Content downloader notebook will download the content for you (either locally or in colab)
- Content will also be in the shared Google Drive (including downloader notebook)



2826097

Homework 1

Part 1 - due Sep 19th

- Written questions – prereqs
 - linear algebra, calculus, and probability
 - Recommend not using AI – you will use these techniques throughout the class
- Coding: basic ML pipelines
 - Pandas, plotly, scikit learn, image transformations
 - Do not procrastinate – the later problems are much harder than the earlier problems

Part 2: due Sep26th

- **No paper this week**
 - we will provide some fun papers to look at on Ed
- Coding: improving models
 - Your model from part 1 is now failing on a secret test set - how can we improve performance without any fancy model techniques?

Fun paper about train/test splits



2826097

Do ImageNet Classifiers Generalize to ImageNet?

Benjamin Recht* Rebecca Roelofs Ludwig Schmidt Vaishaal Shankar
UC Berkeley UC Berkeley UC Berkeley UC Berkeley

Abstract

We build new test sets for the CIFAR-10 and ImageNet datasets. Both benchmarks have been the focus of intense research for almost a decade, raising the danger of overfitting to excessively re-used test sets. By closely following the original dataset creation processes, we test to what extent current classification models generalize to new data. We evaluate a broad range of models and find accuracy drops of 3% – 15% on CIFAR-10 and 11% – 14% on ImageNet. However, accuracy gains on the original test sets translate to larger gains on the new test sets. Our results suggest that the accuracy drops are not caused by adaptivity, but by the models' inability to generalize to slightly "harder" images than those found in the original test sets.

1 Introduction

The overarching goal of machine learning is to produce models that *generalize*. We usually quantify generalization by measuring the performance of a model on a held-out test set. What does good performance on the test set then imply? At the very least, one would hope that the model also performs well on a new test set assembled from the same data source by following the same data cleaning protocol.

In this paper, we realize this thought experiment by replicating the dataset creation process for two prominent benchmarks, CIFAR-10 and ImageNet [10, 35]. In contrast to the ideal outcome, we find that a wide range of classification models fail to reach their original accuracy scores. The accuracy drops range from 3% to 15% on CIFAR-10 and 11% to 14% on ImageNet. On ImageNet, the accuracy loss amounts to approximately five years of progress in a highly active period of machine learning research.

*Authors ordered alphabetically. Ben did none of the work.

ImageNetV2 was **collected in the same way** as ImageNet but ImageNet models **achieve a lower accuracy**



ImageNet

ImageNetV2

visually very similar

Fun paper about train/test splits



2826097

Do ImageNet Classifiers Generalize to ImageNet?

Benjamin Recht* Rebecca Roelofs Ludwig Schmidt Vaishaal Shankar
UC Berkeley UC Berkeley UC Berkeley UC Berkeley

Abstract

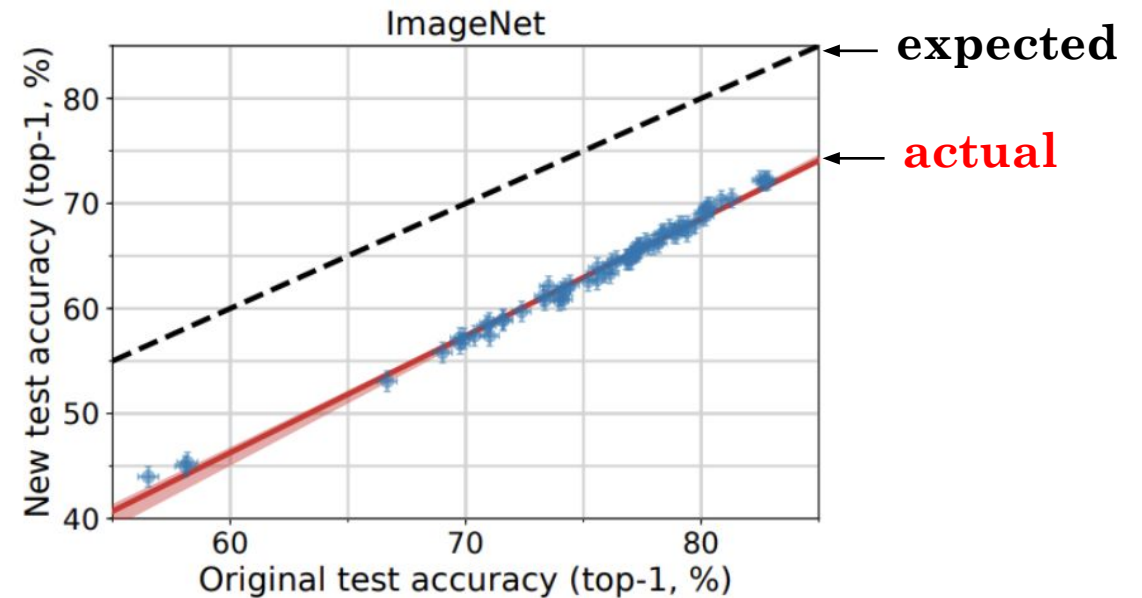
We build new test sets for the CIFAR-10 and ImageNet datasets. Both benchmarks have been the focus of intense research for almost a decade, raising the danger of overfitting to excessively re-used test sets. By closely following the original dataset creation processes, we test to what extent current classification models generalize to new data. We evaluate a broad range of models and find accuracy drops of 3% – 15% on CIFAR-10 and 11% – 14% on ImageNet. However, accuracy gains on the original test sets translate to larger gains on the new test sets. Our results suggest that the accuracy drops are not caused by adaptivity, but by the models' inability to generalize to slightly "harder" images than those found in the original test sets.

1 Introduction

The overarching goal of machine learning is to produce models that *generalize*. We usually quantify generalization by measuring the performance of a model on a held-out test set. What does good performance on the test set then imply? At the very least, one would hope that the model also performs well on a new test set assembled from the same data source by following the same data cleaning protocol.

In this paper, we realize this thought experiment by replicating the dataset creation process for two prominent benchmarks, CIFAR-10 and ImageNet [10, 35]. In contrast to the ideal outcome, we find that a wide range of classification models fail to reach their original accuracy scores. The accuracy drops range from 3% to 15% on CIFAR-10 and 11% to 14% on ImageNet. On ImageNet, the accuracy loss amounts to approximately five years of progress in a highly active period of machine learning research.

ImageNetV2 was **collected in the same way** as ImageNet but ImageNet models **achieve a lower accuracy**



*Authors ordered alphabetically. Ben did none of the work.

Have Fun!!!